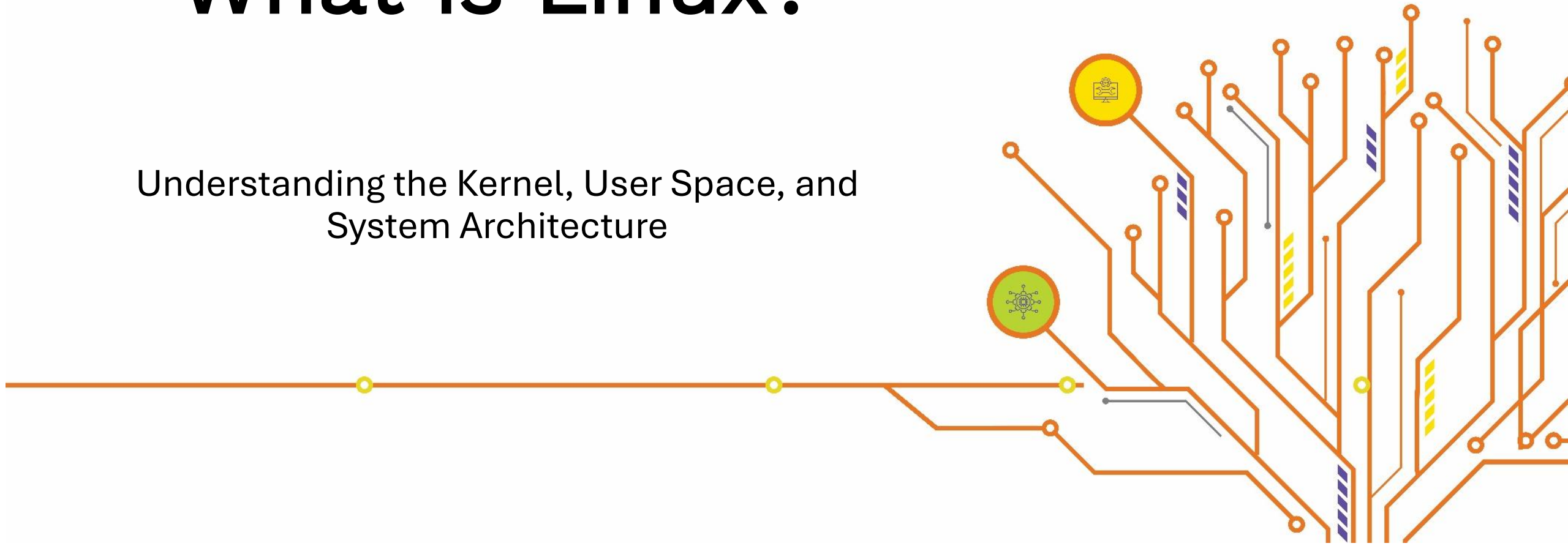
KPIT | APEX LAB

What is Linux?

Understanding the Kernel, User Space, and System Architecture



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Linux - Introduction

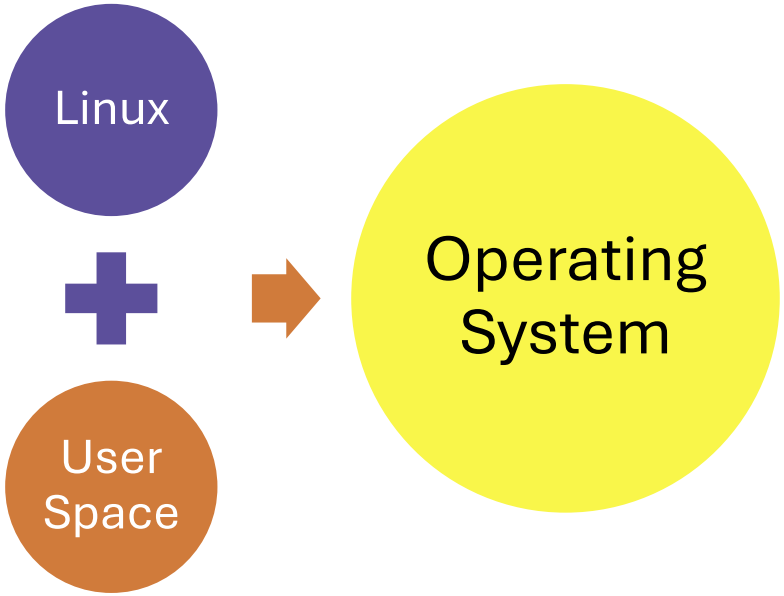
- Linux is a Unix clone written from scratch by Linus Torvalds
- Unix is a multitasking; multi-user computer operating system
- originally developed in 1969 by the group of AT&T employees at Bell Labs.
- Linux and Unix strives to be POSIX compliant
- Open-source kernel

The Linux Philosophy

Do one thing, and do it well	Each program should be focused on a single, well-defined task. Build small, modular programs
Use text as the universal interface	Linux tools typically take text as input and output text.
Everything is a file	Devices, processes, configuration—Linux represents all of them as files in the filesystem
Make programs work together	Output of one program becomes input to another
Configuration over magic	Linux tends to avoid hiding things.
Open systems encourage innovation	Linux's open-source nature encourages contributions, forks, experiments, and learning.

Core Framing: Linux is a Kernel

> "**Linux** is the **kernel**—the privileged core that manages CPU scheduling, memory, filesystems, devices, and system calls."



- A **distribution** bundles kernel + user space + packaging + policies.
- Linux is the kernel; GNU/Linux is the complete operating system; Ubuntu is a distribution.

Important to know!

Kernel ≠ OS ≠ Distribution

- The kernel exposes *syscalls*;
- user space provides *libraries/tools*;
- distributions package them.

Kernel	OS	Distribution
Linux kernel Windows NT kernel XNU (used by macOS, iOS) FreeRTOS kernel QNX microkernel Zephyr kernel	GNU/Linux OS Android OS QNX OS	Ubuntu Fedora Red Hat Enterprise Linux Yocto-based Embedded Linux

Key Features of Linux

Kernel Architecture

- Monolithic kernel
- Device drivers in kernel space
- Loadable Kernel Modules (LKMs)

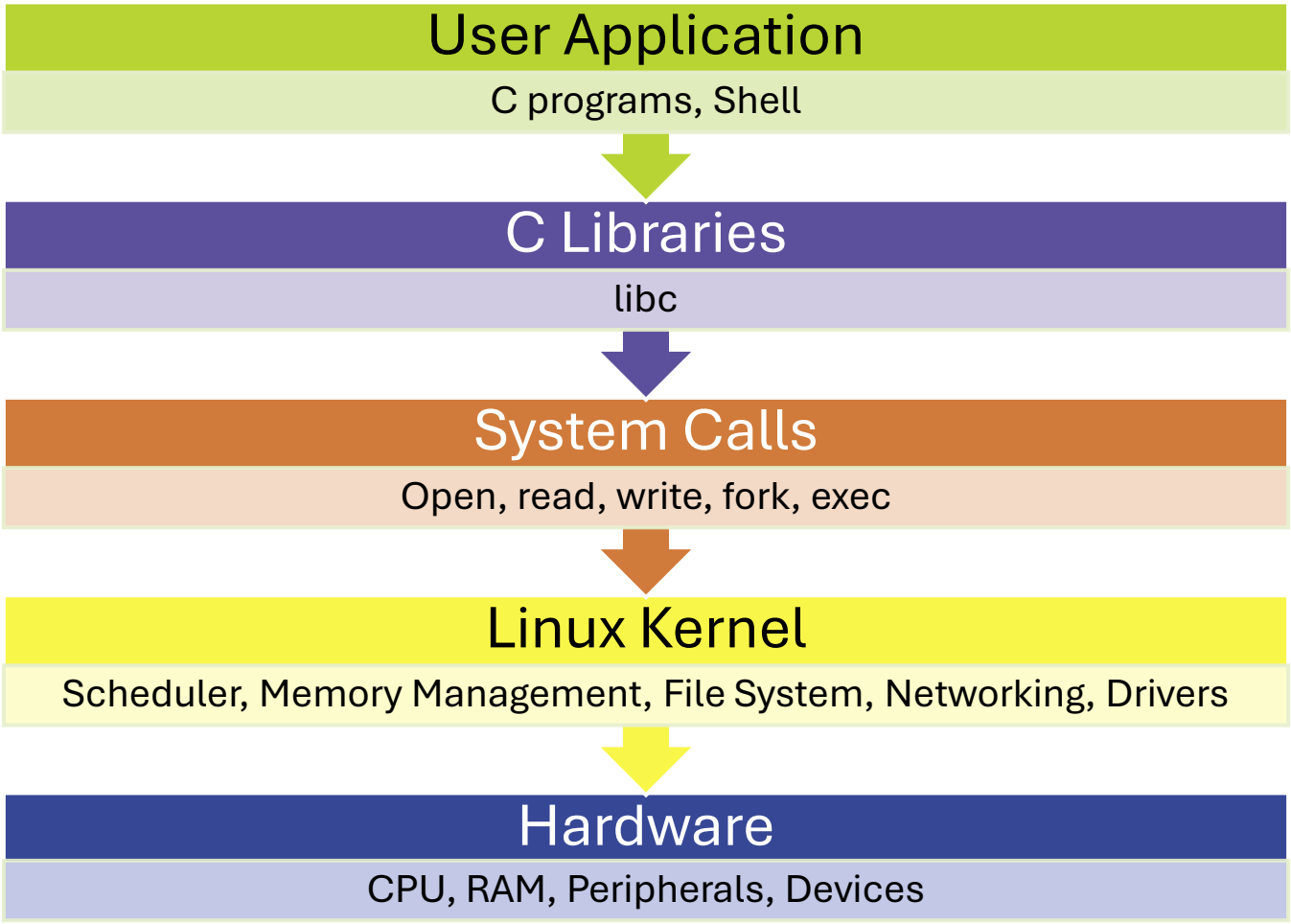
Process & Memory Management

CFS scheduler
Virtual Memory
VFS abstraction

Portability & APIs

Runs on ARM/x86/RISC-V
Syscall interface

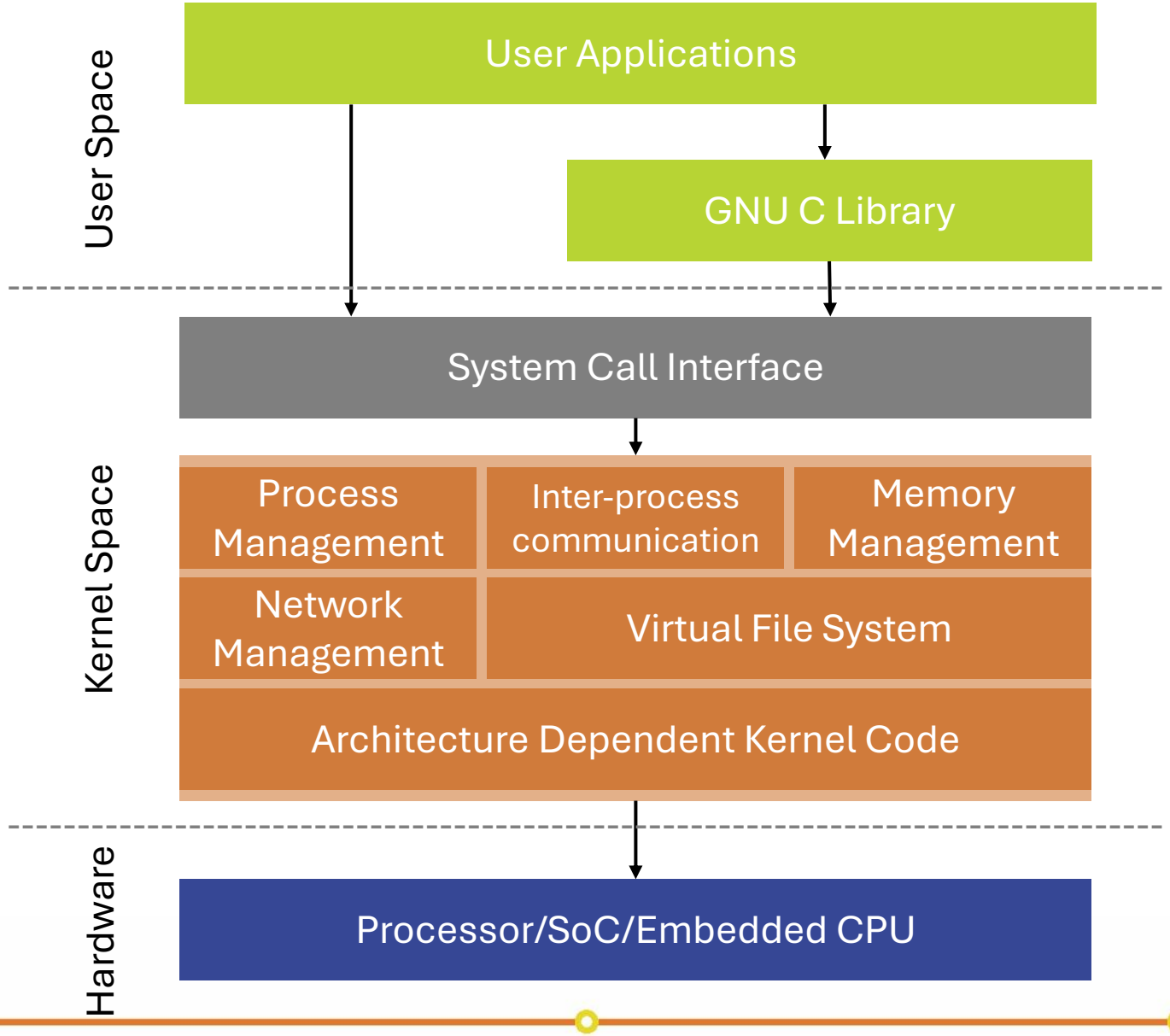
Linux Execution Stack



Linux System Architecture

- Entire memory space is divided into User and Kernel space
- system call interface → act as a bridge
- User Applications doesn't have privilege to access hardware resources directly.
- Kernel executes in kernel space on-behalf of user application

Kernel space = privileged; user space = isolated.



Responsibilities

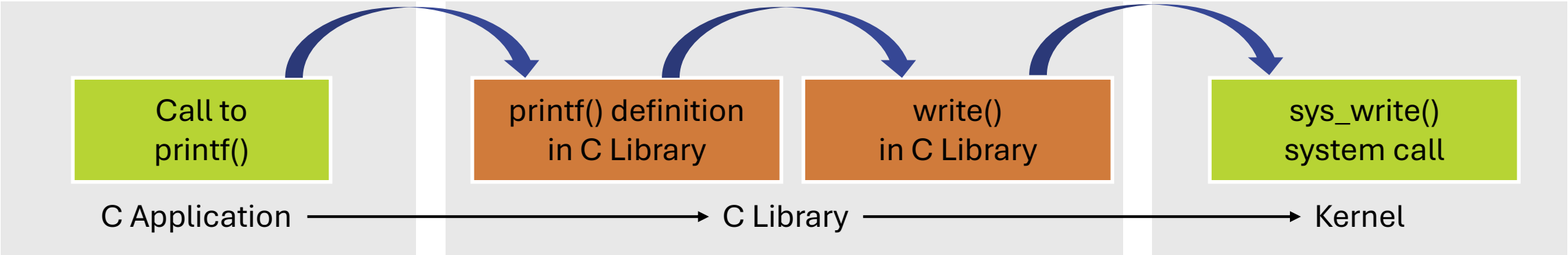
Kernel Responsibilities

- **Process scheduling** (who runs when)
- **Memory management** (virtual memory, protection)
- **VFS** (Virtual Filesystem) and block/FS drivers
- **Device drivers** (character, block, network)
- **IPC primitives** (signals, pipes, sockets)
- **Networking stack**

User Space Responsibilities

- **Libraries** (libc—glibc/musl)
- **Shells** (bash, sh, ash)
- **Core tools** (coreutils or BusyBox)
- **Init systems** (systemd, BusyBox init, OpenRC)
- **Applications & services** (daemons, CLI tools)

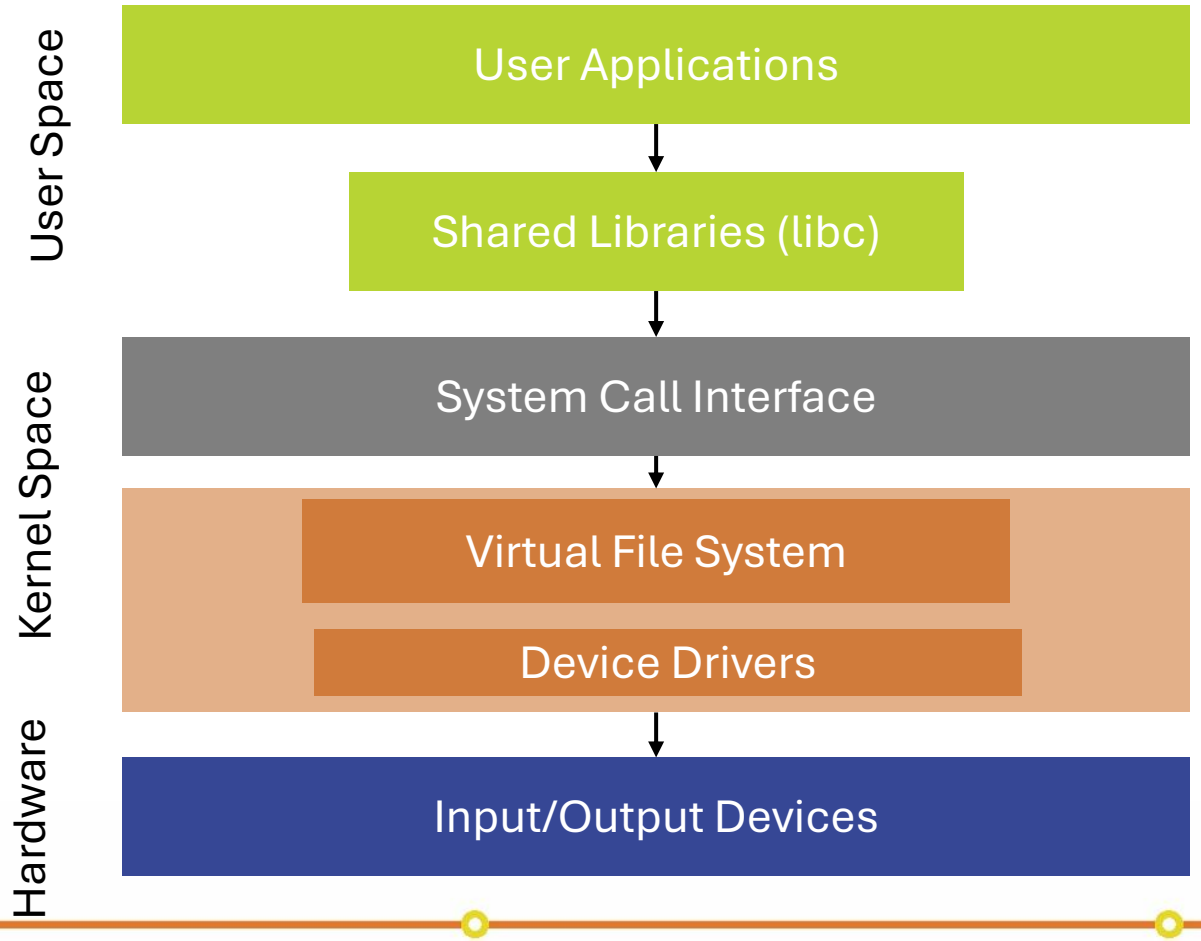
Understanding API, POSIX and C Library



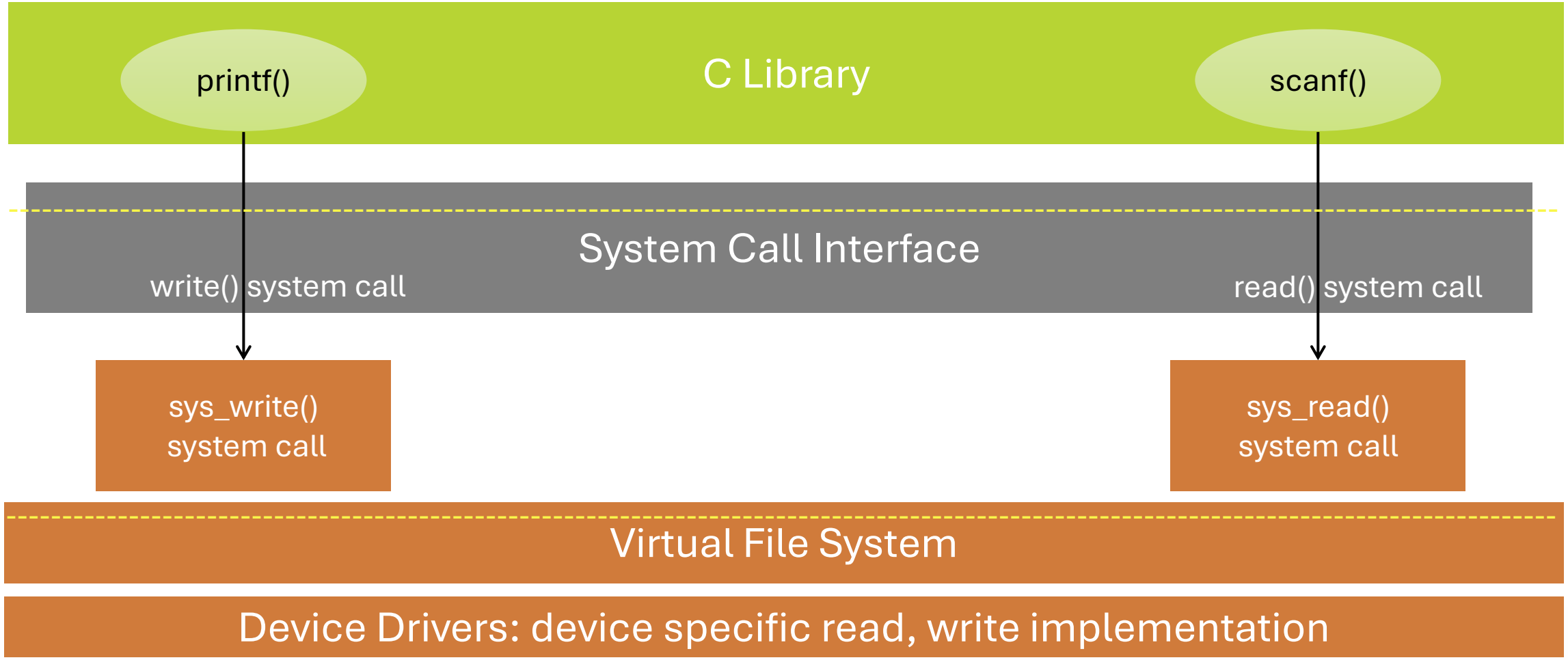
- Applications are programmed against an Application Programming Interface(API), not directly to system calls.
- Application Programming Interfaces in the Unix based OS is based on the POSIX standard.
- The C library implements the main API on Unix based systems, including the standard C library and the system call interface.

Managing I/O devices

- Linux manages I/O devices through a layered architecture that abstracts hardware complexity and provides a unified interface for applications.
- User Space - Applications make system calls like read(), write(), ioctl().
- VFS (Virtual File System) - Provides a unified interface for files and devices.
- Device Driver - Translates generic I/O requests into hardware-specific operations.
- Hardware - Actual device performing the I/O.



An example- I/O devices



User applications and its interface with Kernel

Layer	Example
User app	printf()
C library	write() wrapper
Syscall	sys_write()
VFS	file ops dispatch
Driver	HW-specific write

- Linux recognizes hardware device as file and hence each hardware device is registered with Kernel file system by means of VFS.
- Device drivers implement device specific functions for each system call function and helps device to register with VFS.

Device Drivers

A device driver is a kernel module that allows the Linux operating system to communicate with hardware devices.

User-Space Application

- Calls standard system calls like open(), read(), write().

System Calls → VFS Layer

- VFS acts as an abstraction layer and routes these calls to the correct driver.

file_operations Structure

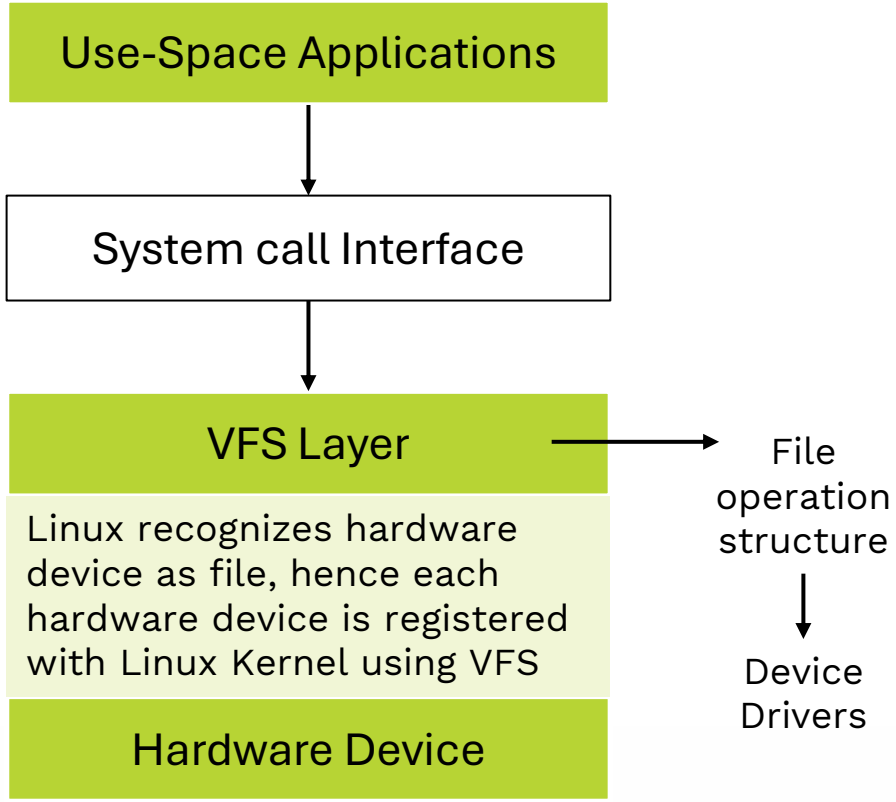
- Contains pointers to device-specific implementations for open, read, write, etc.

Device Driver

- Implements hardware-specific logic and registers itself with VFS using APIs like cdev_add() and device_create().

Hardware Device

- Actual physical device accessed through driver logic.

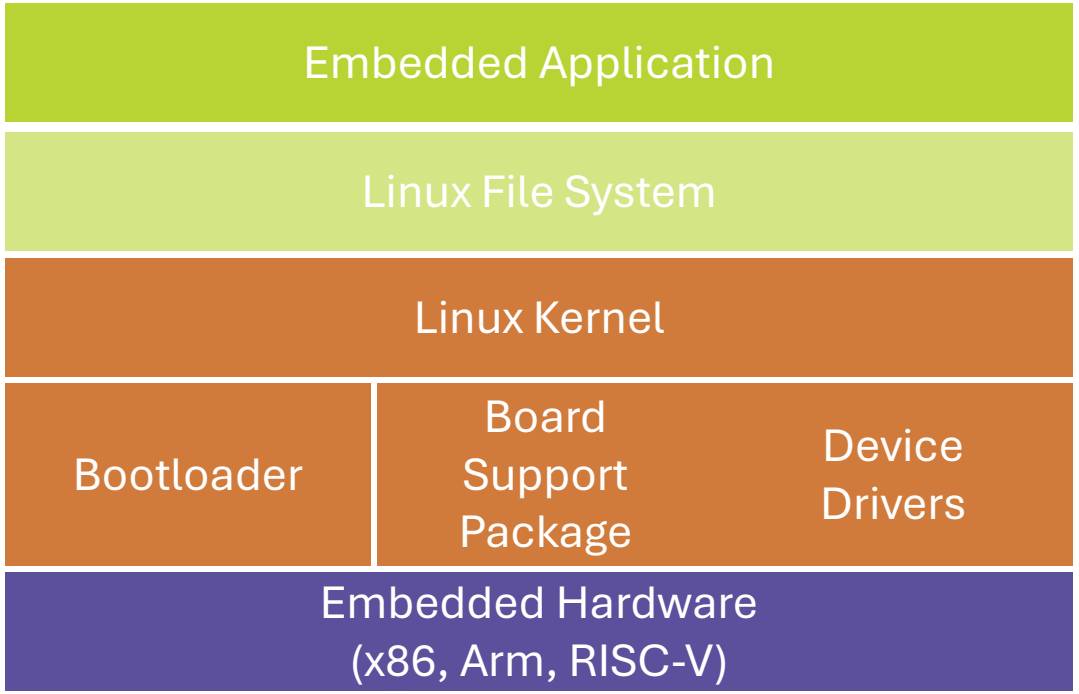


Understanding Embedded Linux Architecture

- Embedded Linux is built around the same Linux Kernel which is apart of most of Linux OS and is available under https://kernel.org/.
- Requires additional packages beyond the original Linux kernel which depends upon the embedded use case.

Creating Target Linux System for Embedded Devices:

- Identify Various Components as per Embedded Use case.
- Configure and Build the Linux Kernel for specific architecture
- Build Root file system
- Setup boot loaders, BSPs etc.



Distributions: Ubuntu vs Minimal Embedded Rootfs

A distribution = kernel + userland + packaging + policies.

A curated collection of:

- **Kernel**
- **User space** (libc + tools)
- **Package manager**
- **Init system & policies**
- Configuration defaults

- **Ubuntu/Debian**: general-purpose, rich tooling, package manager (apt), larger footprint
- **Yocto/OpenEmbedded**: build system to craft custom embedded distributions

Summary

- Linux is a Kernel (Kernel ≠ OS ≠ Distribution)
- Linux = Kernel + Syscalls + Drivers
- Applications run in user space using libc → syscalls → kernel to access hardware.
- User space = libc + tools + apps
- VFS makes everything a file, Devices, processes, configuration — accessed via VFS abstractions.
- Embedded Linux = Bootloader + Kernel + Rootfs + BSP
  - Built using: bootloader + kernel + rootfs + drivers/BSP.
  - Distributions like Ubuntu = general-purpose; Yocto = custom minimal rootfs